Where Model-Based Agents Fail on ARC-AGI-3: A Stage-wise Diagnostic Audit

Thomas Serval — NEOMA Business School; Baracoda AI Labs

Abstract

Why do model-based agents fail on unfamiliar interactive environments? We introduce a stage-wise diagnostic audit for ARC-AGI-3 that locates failures across recognition, transition modelling, goal inference and search, through oracle comparisons and single-stage interventions. On the first three levels of 20 public development games, our symbolic engine, which learns dynamics from its own actions, wins 12 of 60 levels; given source-derived exact rules, our best tested bounded search wins 27, against 39 for a local LLM agent without game-specific knowledge. These comparisons are diagnostic bounds, not controlled estimates of algorithmic superiority. On five quarantined public evaluation games the engine wins no first-three level, and its 19 openings inside the hybrid stop before transition learning in 18 cases — nine at perception, nine at structure recognition — with one defeat: recognition is the first failure point, not necessarily the only one. On 104 synthetic development games, supplying near-perfect recognition raises consistent theories about 2.3-fold without raising the levels won by the engine's own interventions, separating representational progress from gameplay. A declarative language replays all 4,127 observed transitions of the public ARC-AGI-3 games and, in a separate corpus, 177 human-made PuzzleScript games — expressiveness, not rule discovery — and makes representation sufficiency, action independence and theory distinguishability checkable; two simple propositions say when a stage repair can show in level counts and when a symbolic component should be gated into a larger agent. Our linked hybrid submission, ROP5, scores below its LLM-only counterpart on Kaggle's hidden games (22.55 vs 27.34 and 29.89), with no established held-out native-score advantage (45.50 vs 42.98). Better models, faster search and better predictions did not make a better agent: recognition must be validated first, and every repair judged by completed levels. Code, data and preregistrations are released under MIT-0.

1. Introduction

Frontier LLMs now solve the public ARC-AGI-3 games by writing executable world models and checking them by replay; local agents play through a REPL inside the Kaggle notebook. A third, older route — learn a theory of the game in a formal language, then plan in it — promises sample efficiency and verifiability. We built engines of that kind, gated one inside a local LLM agent, and submitted the hybrid. This paper turns that attempt into a method: a stage-wise audit that tells, with measurements, *where* a model-based agent breaks and which repairs are supported by evidence. Its main finding is that an agent can have an expressive language and sound checks and still win nothing on a new game, because its recognition stage — here written on development games — builds no usable representation before reasoning starts; and that repairing recognition alone moves the failure downstream rather than producing wins.

Contributions. (1) **Diagnostic methodology (§4–5):** oracle comparisons that bound what each stage can explain, and a seven-step procedure that turns a lost level into a demonstrated defect, an intervention-supported cause or an open hypothesis; any agent with logged stages can apply it. (2) **Formal verification (§3):** a declarative language whose declared read/write sets make representation sufficiency, action independence and theory distinguishability checkable game by game, with two elementary propositions on the chain as a whole. (3) **Empirical findings (§4–6):** recognition is the first bottleneck of our symbolic chain on new games; supplying it, speeding up search or improving predictions did not translate into levels won; and stagnation gating cost levels — documented with a loss map, a complete audit case and an audited hybrid, all released. We joined the competition late — work started on 30 September 2026, first Kaggle submission on 18 October — so all of this was built and measured in about five weeks, largely by LLM coding agents under human direction. The engine calls no LLM; the agent around it does.

2. Prior work

Frontier systems build world models with cloud LLMs. Tycho (Lehmann, Aioanei and Vahdati, 2026) notes that matching transitions does not identify the objective; OPINE-World (Courtis, Li and Sanner, 2026) explores by ontology errors; Kepler (Wu, 2026) finds that animation frames carry information missing from settled grids — the kind of loss our sufficiency check detects; Rodionov (2026a, 2026b) studies executable world models written by coding agents. Twin (Skoutnev et al., 2026) repairs a program against every past transition and infers the goal before any reward on 156 of its 179 cleared levels; AWM (Pan, Tan, Yuan and Yao, 2026) encodes learned rules in Lean 4 and proves sealed levels unwinnable. Both leave screen parsing to the frontier model: once recognition is done, formal models succeed — which is exactly where our local engine fails. DiG-bench (Battleday et al., 2026) shows the converse for LLMs: giving Gemini 3.1 Pro the rules of 70 text games raises its wins from 18 to 69, whereas our planners, given exact rules, stay below an LLM agent that has none. Banwait (2026) argues that perception, not planning, governs ARC-AGI-3 and proposes an explicit spatial representation of the grid; our audit is consistent with that claim and complementary to it — it does not test a better

perception, but shows, by controlled single-stage replacement, where a given agent loses information and what a repair changes downstream; Liew (2026) shows that public games can be reached by trivial heuristics, a caution for any public-set result, including our G50T; a think-aloud study (*How Humans Learn Unfamiliar Worlds*, 2026) finds that people rarely anticipate a mechanism before triggering it (4 %). Lehmann and Vahdati (2026) find that self-improved harnesses gain 8.42 RHAE on development games but 2.80 [−1.44, 7.35] held-out; our engine shows that pattern in extreme form. Wu, Passenger and Song (2026) find that a coding agent's level-specific knowledge transfers poorly, and The Duck (Tufa Labs; ARC Prize, 2026) that hand-crafted tools hurt. EMPA (Tsividis et al., 2021), the Apperception Engine (Evans et al., 2019) and inductive general game playing (Cropper, Evans and Law, 2019) learn theories from play; ASP for games (Thielscher, 2009), unsolvability certificates (Eriksson et al., 2017) and partial-order reduction (Wehrle and Helmert, 2012) supply the formal machinery.

System	Model at play	Runs	Built at play	Checked by	Reported result	Cost
Tycho	frontier LLMs	cloud API	program world model	transition replay	mean 88.49 RHAE, public	API
OPINE-World	Claude Opus 4.8	cloud API	world model, ontology errors	ontology-error signals	1.5 % → 78.40 % RHAE	API
Kepler	Claude Opus 5	cloud API	auditable world model	audit and replay	100 RHAE, public	≈ 778 USD
Twin	frontier coding agent	cloud API	program repaired on all past transitions	full replay	93.3 %, 179/183 levels	API
AWM	Claude Opus 5	cloud API	algebraic model in Lean 4	kernel-checked proofs	40.7 → 99.9, public	API
The Duck	Qwen 3.6 27B	local (Kaggle)	REPL code	—	Milestone 1 winner	Kaggle GPU
ROP5 (ours)	Qwen3.8-Flash-Next 4-bit + LLM-free engine	local (Kaggle GPU + idle CPU)	engine: transition model from own actions	per-action checks; offline stage audit	22.55 hidden; 36.44 public	whole project < 400 EUR, no paid API, no training

Reported metrics are not directly comparable (different sets, models, budgets and scoring). Unlike these systems, our contribution is not a score but a way to locate, stage by stage, why an explicit model-based agent fails, measured in the offline Kaggle setting. To our knowledge no prior ARC-AGI-3 work separates the stages of an agent this way.

3. Approach: systems and theory

One language, two worlds of games. All our engines share a GDL-style declarative language (Love et al., 2008): a protocol (state, legal moves, next state, terminal, goal) with five universal constraints; 17 mechanism contracts in the spirit of VGDL (Schaul, 2013), each declaring roles, parameters, read and write sets and the move that tests it; and the move budget as an axiom (exhaustion is a defeat). Seventeen contracts cover the 20 public games; a wider catalogue of 27 mechanisms (with frequencies estimated on PuzzleScript and click puzzles) and 7 goal forms serves as the induction prior. One exact program per game reproduces **4,127 of 4,127** observed transitions of the 20 public ARC-AGI-3 games (no-rule baseline: 420). One interpreter written in the language, taking each game's rules as data, replays **177 of 177** human-made PuzzleScript games — **38,924 of 38,924** moves and 852 of 852 level ends against the reference engine (no-rule baseline: 23 % of moves, 0 ends); the corpus is the deterministic subset that engine accepts, and two games are exact only up to a 30-minute cap; an earlier interpreter version was exact on fewer games, and all 177 became exact after a fix to how rule properties are bound (repository commit history). This interpreter and corpus are released as a test bed for other world-model learners. The exact programs were written by reading the games' sources: they show what the language can express, not what an agent can learn, and serve only as oracles.

Online engine. Written in Python, it perceives cells and objects, learns a transition model (object pushes, player motion) from the frames that follow its own actions, with candidate mechanisms proposed by an inventor module, and plans over that model. A separate **research inducer** fits declarative theories to recorded play with answer-set programming (ASP) offline.

LLM agent (RO) and linked submission (ROP5). Franzen's public notebook (github.com/da-fr/arc-agi-3-solution, Apache-2.0) runs a local quantised model (albucino/Qwen3.8-Flash-Next-W4A16-FP8PLE on SGLang) through a Python REPL; RO adds deterministic tools after 40 actions in a level (map of observed states, paths, surprise-checked traversal, memory of fatal actions). ROP5 adds a gate: at the start of a game, then at most once per level after 40 actions without progress, the engine opens in an isolated child process (at most ten at a time; the LLM never waits). The parent alone owns

the game and re-checks deadline, terminal state, offered actions, level, observation number and frame hash before each engine action; budgets are bounded (150 actions per level; 90 s wait and 300 s wall per opening; stop at the fourth repetition of an action from the same frame; no reset after a game-over); at the end of every opening the LLM receives a summary of the engine's actions. Thresholds were set on public development games. The engine runs on idle CPU cores (the 20 public games in 74 s of CPU) while the LLM uses the GPU (48 minutes on an RTX PRO 6000 for 20 games).

Component	Data it sees	What it computes	What it produces
Online engine (in ROP5)	frames after its own actions	perception, transition model, plan	actions; logged stop reason
Research inducer (offline)	recorded play	ASP theories fitted to transitions	theories, predictions, loss map
Language + exact programs (oracle)	game sources, read by the author	replay, read/write sets, planning	exact replays, independence proofs, planner results
Sufficiency detector (offline)	recorded traces	collisions of represented states	counter-examples with coverage

Only the online engine plays inside the submission.

Representation. Let X be the set of observations (a grid of up to 64×64 cells in 16 colours, with the level counter and the offered actions) and S the set of represented states: finite assignments to the fluents of the language (objects with identity, type and cells; counters; flags). A representation is the partial map $\varphi: X \to S$ computed by the recognition stage; it is *undefined* on x when no controlled object or no structure is found — the engine's two recognition stop reasons. A transition model is a map $f: S \times A \to S$, and it is judged against $\varphi(x_+)$ for each observed transition (x, a, x_+). φ is *sufficient on a trace* if any two observed transitions with $\varphi(x) = \varphi(x')$ and the same action have $\varphi(x_+) = \varphi(x'_+)$, that is, the observed dynamics factor through φ. Recognition fails when φ is undefined or not sufficient; all later stages see the game only through φ.

Properties. Write *R(a)* and *W(a)* for the fluents of S read and written by the contracts an action a triggers; by inertia, a changes nothing outside *W(a)*, and which contracts it triggers, and what they write, depend only on the state restricted to *R(a)*.

Proposition 1 (sufficiency). If frames $x \neq x'$ have $\varphi(x) = \varphi(x')$ and the same action yields $\varphi(x_+) \neq \varphi(x'_+)$, every predictor *f(φ(x), a)* errs on one of the two transitions, whatever its language, data or budget. □ Such an error can be removed only by a richer φ, a longer history or more observation. An observed collision refutes sufficiency; no collision does not certify it, so coverage of revisited state–action pairs must be reported. *Check:* on KA59 three such pairs differ by 18 game pixels the sampler never reads.

Proposition 2 (independence). If $W(a) \cap R(b) = W(b) \cap R(a) = W(a) \cap W(b) = \varnothing$, a and b commute in every state — the Bernstein condition behind partial-order reduction — including the shared move budget while its threshold is not crossed (Appendix A). *Check:* on 311 action pairs of the exact programs, the 24 predicted independent all commute (0 false).

Proposition 3 (distinguishability). Theories consistent with all observations can be told apart only by a sequence, from a reachable state, where they predict different observations; if they differ in one contract, every separating sequence triggers that contract in at least one of them. □ With binary outcomes, at least $\lceil \log_2 |H| \rceil$ experiments are needed in the worst case to identify the theory, though winning does not always require it. *Check:* on G50T level 6 a latch and a plate-triggered mover explain every frame; the language returns the single separating move.

Proposition 4 (a level is won only through the whole chain). Let a level be won if and only if stages 1…K (recognition, model, goal, search) all succeed, and let p_k be the probability that stage k succeeds given that stages 1…k−1 did. Then P(win) = p_1 p_2 … p_K by the chain rule, and an intervention that raises p_j by Δ while leaving the other conditionals unchanged raises P(win) by $\Delta \cdot \prod_{i \neq j} p_i$. □ When another stage almost always fails, even a large improvement of one stage is invisible in level counts; with p_1 = 0 the chain wins nothing. In practice repairing one stage changes who reaches the next ones, so the proposition is a reading guide rather than a prediction; §4 shows the pattern it describes.

Proposition 5 (when a gate pays). Let a gate open the engine on a game; with probability r the engine recognises the game and gains g levels over the LLM alone, and opening costs c levels through overhead or interference. The expected net gain $r \cdot g - c$ is positive iff $r > c/g$. Gating on a free recognition test that passes with probability s when the engine would recognise the game and f otherwise gives $r \cdot s \cdot g - (r \cdot s + (1-r) \cdot f) \cdot c$, which is non-negative iff $f \leq r \cdot s \cdot (g - c)/((1 - r) \cdot c)$: the false-positive rate a gate can tolerate shrinks to zero as recognition becomes rare. □ A gate triggered by stagnation alone has f = 1 and loses whenever $r < c/g$.

These properties are elementary; their value is that the language makes each of them checkable game by game, through explicit contracts and recorded witnesses, and that together they localise failures across a staged agent.

4. Results I — oracle comparisons

Protocol. ARC-AGI-3 has no separate training split; its 25 public games form the public set. We used 20 of them for development, so results on them are in-sample and reported as such; out-of-sample evidence comes from a quarantined evaluation set of five public games (vc33, tu93, wa30, s5i5, cn04; 39 levels) and Kaggle's hidden set. The five were fixed on 17 October to cover all three interaction modes and to resemble hidden games rather than our successes (two were blocked for the baseline agent). Their exposure is disclosed: before quarantine, an automated catalogue had read all 25 game sources, these five included, to list primitives (those primitives are marked and not counted as coverage); the baseline agent had played all public games; and information about one game reached the team. After quarantine no source, trace or film of these games was read, and no rule, setting or threshold was chosen from them. **Scores.** Native ARC scores follow the official method: per level, $(\min(\text{human actions} / \text{agent actions}, 1))^2$; per game, the average of level scores weighted by level index; overall, the mean over games. Metric: levels won in the real game, plus native ARC scores. Held-out runs were preregistered before they were run (repository, `docs/preregistration/`): the engine on 23 October (one authorised pass after a first attempt was interrupted at the start of cn04), the LLM agent on 3 November and the hybrid on 4 November (two passages each, same model and weights as Kaggle).

Rung	Start state	Budget	Runs	Gating
(a) Engine	game reset	150 actions per level	1	sequential
(b) Exact rules + portfolio planners	true start of each level	60 s per level	1	independent
(b′) Exact rules + goal-directed search	true start of each level	1 M nodes, 300 s per level	1	independent
(c) LLM agent (RO)	game reset	48 min for 20 games, 10 streams	2	sequential
(d) Hybrid (ROP5)	game reset	as (c)	2	sequential

The per-game best of (a) and (c) is a retrospective bound chosen after seeing the results, not an executable system. Under sequential gating a level is attempted only after the previous one is won, so the engine's 0 of 15 held-out first levels means it failed level 1 of each game. Budgets, algorithms and gating differ, so each comparison is a diagnostic bound, not a causal decomposition:

Comparison	What it allows us to conclude
Engine vs exact rules + search	how far the whole learned chain falls short of source-derived knowledge, not which stage causes it
Exact rules + search vs LLM agent	the limits of the search we evaluated, under different budgets
RO vs ROP5	the overall effect of stagnation gating
Local repair vs its off-switch	the causal effect of that repair on the metrics tested

Rung	First 3 levels (/60)	All levels (/144)	Held-out first 3 (/15)	Held-out all (/39)
(a) Engine	12	18	0	0
(b) Exact rules + planners, 60 s	18	—	—	—
(b′) Exact rules + goal search, 300 s	27	—	—	—
(c) LLM agent (RO)	39	60	15	24
(d) Hybrid (ROP5)	43.5	68.5	11.5	23
Per-game best of (a) and (c)	—	69.5	—	—

Wilson 95 % intervals treat levels as independent and overstate precision: engine 12 [7, 19], planners 18 [12, 26], goal search 27 [20, 35], agent 39 [31, 46] of 60; held-out, engine 0 [0, 3] and agent 15 [12, 15] of 15. Game-bootstrap intervals for search and agent are [17, 37] and [31.5, 47].

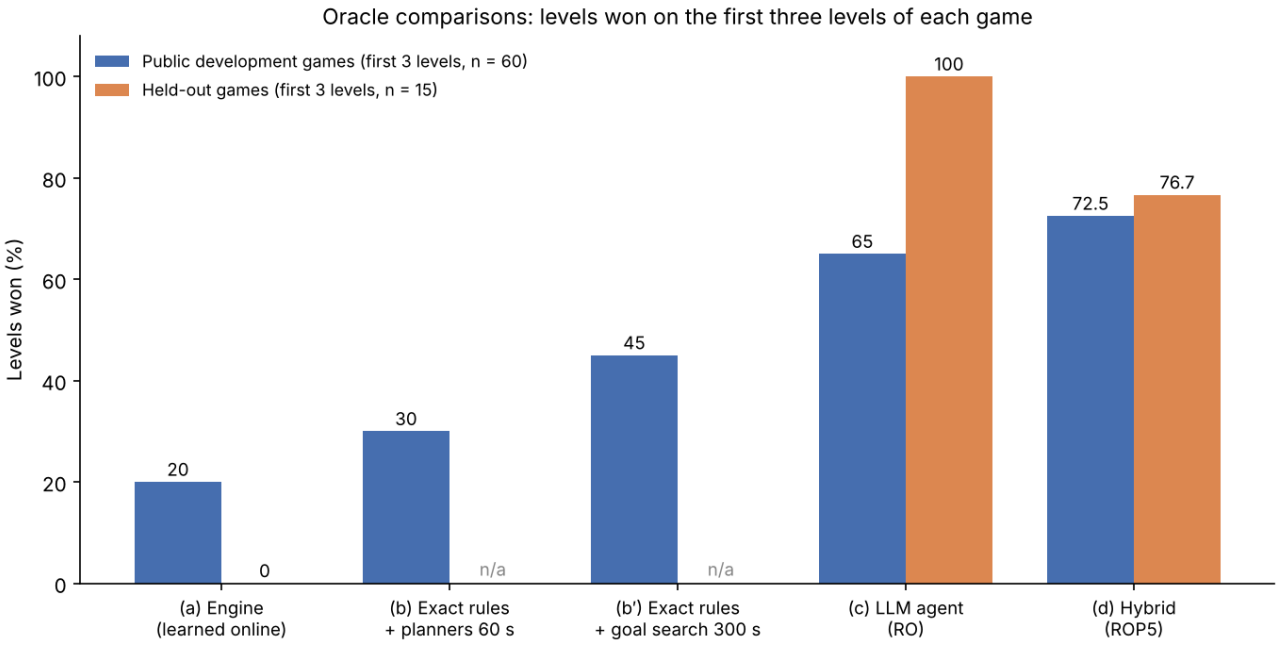


Figure 1. Oracle comparisons: levels won on the first three levels of each game, development (n = 60) and held-out (n = 15). Rungs differ in budget and gating.

The symbolic pipeline does not carry over. Six of the engine's 12 early levels are on G50T and KA59, the games it was developed on; on the other 18 development games it wins 6 of 54, so its 0 of 15 held-out is not a significant drop by itself ($P \approx 0.17$ at that rate), but it never wins a level on a new game. The engine's 19 openings inside the hybrid on the held-out games (two passages; we did not analyse the stop reasons of the standalone run separately) show where it stops: nine with *perception not identified* (after one probe per action it could not find which object the actions move), nine with *structure not recognised* (the recogniser, written on development games — its message names G50T — found no point-like goal), one with a game-over. No hypothesis was generated, so the transition learner, inventor and planner were never reached. The sufficiency check on these short traces (54 engine actions, at most four per opening) found no collision and had no power, so this is a recogniser failure, not a demonstrated violation of Proposition 1. Read by stage: recognition failure is demonstrated; transition learning was never reached there and is untested; search failure is demonstrated only with exact rules.

A larger synthetic check. On 104 procedurally generated games outside ARC (81 keyboard mazes, 23 click tile puzzles; our own generator, used during development, so not held out), a version of the chain from 25 October (80 actions and 240 s per game) won 5 first levels (Wilson 95 % [2.1, 10.8] %), stopped without an admissible action on 57 and lost 42; all five wins are keyboard games, and on the click games it made no initial probe and won nothing (0 of 23). The dominant refusals in its logs are "transitions needed" and "no matched mobile object": the same bottleneck on about twenty times as many games. By contrast, our LLM-free explorer, which explores without building a model, wins 101 to 102 levels in total on the same suite: the mazes are solvable by exploration alone, and it is the model-based route that stalls at recognition.

Supplying recognition moves the failure downstream. On the same synthetic suite we injected the recognition the chain lacks, first from an automatic detector of the controlled object (correct on 161 of 162 cases, a single control mode), then from an oracle giving the player's exact colours. The number of situations in which the chain held a consistent theory rose from 72 to 106 with the detector and to 165 with the oracle (70, 103 and 164 on a second seed), and plans authorised for play from 41 up to 87, with no theory lost. In play, with two detector variants (first candidate colour, all candidate colours), the engine's interventions rose from 36 to 44 and 56, every additional intervention diverged from the real game within its first actions, the levels won by the engine's own interventions stayed at 8 (the explorer it ran inside won the others), total synthetic levels changed by +1 and −2, and on the 20 public games no level changed. This is the pattern Proposition 4 describes: raising the first stage did not raise wins because the next stage, plan fidelity, then failed. It was measured on development data with an earlier component, before the paper was planned, so it illustrates rather than tests the proposition.

Knowing the rules is not enough. With exact rules, our best search wins fewer early levels than the LLM agent (27 vs 39; 23 for the same search on a slower machine). As exploratory level-wise tests (levels of one game are not independent), on the levels won by exactly one of the two the agent wins 16 to 6 (exact sign test, $p = 0.052$) and 19 to 5 ($p = 0.007$) against each RO passage, and 15 to 9 ($p = 0.31$) counting a level for RO only when both passages win it. The clustered estimate resamples the 20 games with replacement (20,000 replicates, percentile interval), scoring RO by the mean of its two passages per game: the agent's total advantage is 12 levels (0.6 per game), 95 % [1, 23.5]; per game, the agent is ahead on 10, the search on 5, tied on 5. This does not show that LLM reasoning is intrinsically superior to symbolic search: it shows that the best search we evaluated, even with exact rules, stayed below the LLM agent in the tested configurations. The

levels the agent wins and the search misses are not longer (median human solution 32.5 actions, against 40 for those the search wins). Pruning by Proposition 2 changes nothing on blocked levels, where the 24 proved pairs never occur; an extended commutation rule for clicks (no counterexample in the runs, not proved) cuts generated nodes by 45 % on FT09 and 77 % on TN36 (TN36 level 1 in 0.6 s instead of 24.1 s) but wins no new level, nor does a compiled simulator 1.66 times faster. What the search lacks is not speed; we hypothesise it is knowledge of which sub-goals to pursue.

The hybrid helps where the engine was built and costs elsewhere. On public games ROP5 obtains 68.5 levels against 69.5 for the per-game best of engine and agent, and 36.44 against 30.73 for RO in native score (paired +5.71, game-bootstrap 95 % [−4.02, +17.81]); G50T and KA59 contribute +7.05 points to that mean and the other 18 games −1.35. In both public passages the engine alone finished G50T, all 7 levels in 360 actions without a single LLM token, where RO won 0 and 2 levels — what the engine does once it recognises a game, not transfer. On the held-out games ROP5 averaged 45.50 against 42.98 for RO (preregistered prediction ROP5 ≤ 42.98, refuted in direction; three games up, two down, sign test p = 1.0) and completed 23 levels against 24. It finished vc33 twice, which RO never did, but lost levels on cn04 (1 of 6 twice, against 4 of 6 twice) and wa30 (1 and 2 of 9, against 3 twice), so it won 11.5 held-out first-three levels against 15. The engine played 54 of 2,657 actions there and won no level during its openings; on cn04 it played one and two actions, yet the LLM made fewer tool calls in the same time (123 and 124, against 156 and 136 for RO). Overhead — child processes, per-action checks, the summary added to the context — is a plausible cause we did not isolate. By Proposition 5 this is what a stagnation-triggered gate should do on new games, where no opening recognised the game (0 of 19: 18 stopped at recognition, 1 ended in defeat; Wilson upper bound 0.17, or 0.43 counting games): a gate requiring recognition first would never have opened there and, if its test is free, would have matched RO by construction, while still opening on G50T. On Kaggle's hidden games ROP5 scored 22.55 against 27.34 and 29.89 for two identical RO submissions, too few runs to establish an effect (a 0.00 score of the base notebook on another date was a run that failed at start-up, not variance).

Other negative results. A pinned factual summary for the LLM agent scored 24.82 on Kaggle, and an executable Python model of the game showed no gain with a small model. In an AlphaEvolve-style loop, thirteen LLM-proposed changes to the research inducer, under a fixed judge with identity and empty-theory controls, reached at best 1.018 times the baseline against a preregistered threshold of 3; removing all learned laws dropped the metric from 0.250 to 0.110, so the judge could detect change.

5. Results II — where the chain breaks

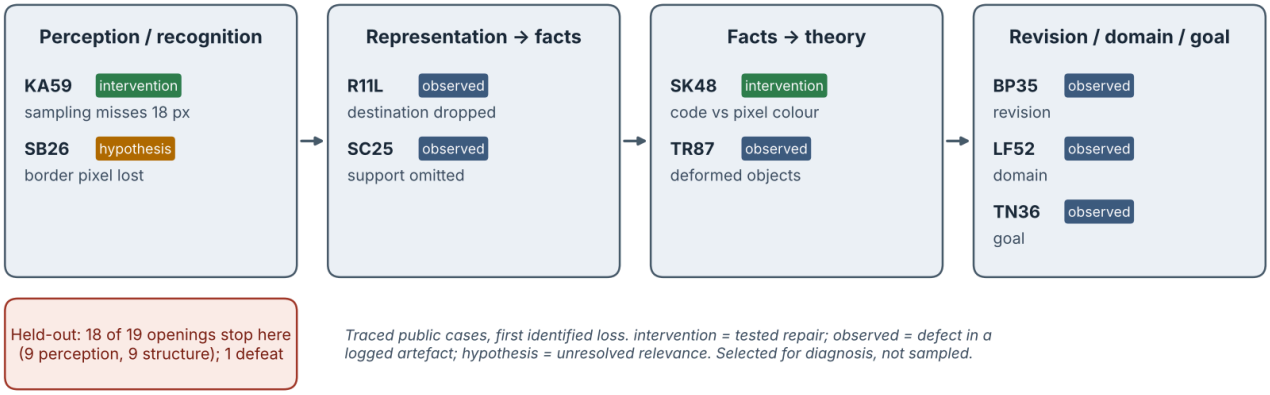


Figure 2. First identified loss for the traced public cases (observed, intervention or hypothesis; see table), and the stage where the held-out openings stopped.

Game	First failing stage	Evidence	Status
KA59	Perception (sampling)	same state and action, different successors; 18 pixels never sampled	intervention
SB26	Perception	border pixel lost; relevance unproven	hypothesis
R11L	Facts	destination kept in objects, dropped from ASP facts; guards block 3 of 5 events	observed
SC25	Facts	newly visible support omitted	observed
SK48	Facts → theory	object code compared with pixel colour: 20 false change facts, a 131-cell law on identical images	intervention
TR87	Theory	faithful input; deformed objects counted present without support	observed
BP35	Revision	hypothesis rejected by an observation outside coverage	observed
LF52	Domain	marker attributed to the wrong object; engine refuses to act	observed
TN36	Goal	correct predictions, uninformative goal	observed

Observed means a defect in a recorded intermediate artefact; *intervention*, a tested local repair with a measured downstream effect; *hypothesis*, unresolved relevance. The nine cases were selected for diagnosis, not sampled to estimate prevalence.

Audit procedure. Inputs are a frozen agent version, staged traces and, when available, a trusted reference for one interface. (1) Record the inputs and outputs of every stage; (2) look for a local counter-example (equal inputs with different outcomes, a fact contradicting the frame, a law contradicted later); (3) find the first transformation where the needed information disappears; (4) replace only that transformation with a controlled reference (finer sampling, exact facts, exact rules); (5) replay the downstream stages unchanged; (6) measure the repair against an off-switch control that must reproduce the unrepaired run exactly; (7) classify the outcome as *defect demonstrated*, *cause supported by intervention* or *unresolved hypothesis*. Freeze the intervention before testing it, keep contradictory cases, and separate prediction gains from action selection and level completion.

A complete case, from detection to gameplay. Figure 3 follows one repair through all steps on our LLM-free explorer, the component that preceded the online engine. On recorded play of the 20 public games (38 levels), 339 state–action pairs were revisited at least twice; 24 had two different successors — Proposition 1 collisions — in 4 games only. Reading the successors located the first lossy step: the state key kept a gauge glued to the screen border (sc25: 8 pixels in columns 62–63; sb26: 2 pixels), which the 1-pixel border mask missed; ls20 and g50t pointed instead to a hidden step counter and a path memory. The repair replaced only that step, masking a 3-pixel band (two lines of code). Switched off, it reproduced the reference action for action; switched on, no level was gained or lost (19 and 19 public levels over two seeds; 101 and 102 levels in total on the synthetic suite, whose mazes the explorer solves by exploration alone), and tn36 level 1 was won in 8 actions instead of 75 and 79 (human baseline 32): a development gain in efficiency, with every step of the procedure shown.

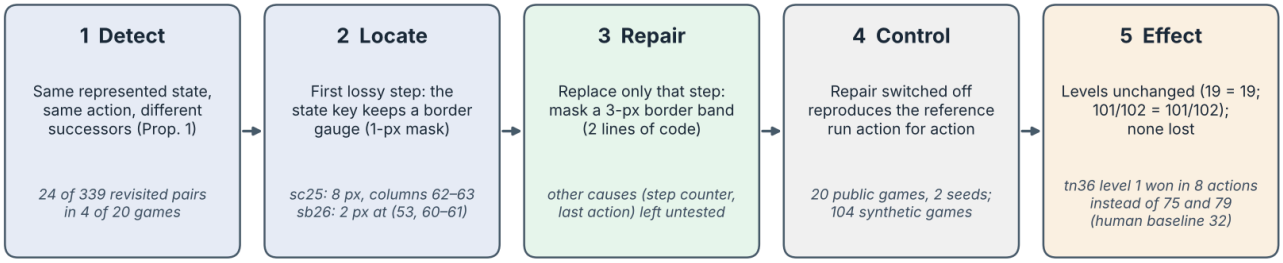


Figure 3. One audit case end to end (LLM-free explorer, public games): detection by a Proposition 1 collision, location of the lossy step, single-step repair, exact off-switch control, and effect on play.

Intervention	Observed effect	Limit
KA59: finer sampling when the detector fires	visible free-rollout predictions 5 → 8 of 13	reused data; one non-visible prediction lost; no level gain
SK48: decode the object code before comparing it with a pixel colour	20 false change facts removed; exact prefix predictions 4 → 8 of 30	22 prefix errors remain; no free-rollout gain
TN36: extended commutation pruning, same planner and budget	level 1 planned in 0.6 s instead of 24.1 s; 77 % fewer nodes	rule not proved; no extra level at 60 s or 300 s

All repairs pass an exact off-switch control; they are development gains, not gameplay gains.

6. Results III — what the language reveals about the games

Laws that held and a law that failed. ARC-AGI-3 environments are designed to use only core-knowledge priors (Chollet, 2019; ARC Prize Foundation, 2026), and the language makes some of them testable. Five candidate universal laws — some protocol conventions rather than discovered regularities — hold in every public game: one action per step; never a level end and a defeat on the same step; one anchor per rigid object; inertia; nothing but a reset after a defeat. A sixth, "nothing overlaps a solid", is false in 16 of 20 games and became a parameterised mechanism.

Rule size tracks search difficulty more than LLM difficulty. Exact programs range from 795 (FT09) to 12,062 (BP35) nodes. Program size correlates with the early levels the goal-directed search wins (Spearman $\rho = -0.64$, $n = 20$, $p < 0.01$) but only weakly with the LLM agent's success ($\rho \approx -0.3$, not significant). Size depends on the encoding and its author, so it is not an intrinsic difficulty measure.

7. Discussion

What the engine does contribute. Where it recognises a game it is efficient and cheap: on G50T and KA59 it wins 12 of its 18 levels (6 of its 12 early levels) where RO wins 2.5 of those levels on average over its two passages, finishing G50T alone with up to 5.6 times fewer actions than the human baseline, on idle CPU cores. Every action is checked before it is played, and every opening records exactly where it stopped — the logs that made this audit possible.

Recommendations for builders of model-based agents. (1) Test recognition on games outside the design set before trusting an inducer or tuning a planner; report its success rate. (2) Search recorded play for representation collisions and report revisit coverage. (3) Validate every repair by its effect on levels won, against an exact off-switch control; a better prediction is not a better agent (Proposition 4). (4) Open a symbolic component only after a recognition test with few false positives, not on stagnation alone (Proposition 5). (5) Do not expect faster search to replace the priors that tell a planner which sub-goals matter. (6) Prefer a frugal, offline loop for diagnosis: this whole programme cost under 400 EUR of rented GPU, with no paid API and no training. Weight adaptation (Akyürek et al., 2024; Zweiger et al., 2025) is a separate hypothesis we did not test.

Limitations. Recognition is the first failure point we observe on new games, not necessarily the only obstacle to generalisation: the stages after it were never reached there. Development data were reused for diagnosis; exact rules were written from game sources; rung budgets differ; held-out results rest on five games with disclosed incidental exposure and a pod reproduction; Kaggle comparisons are few unpaired submissions; corpus replay establishes agreement on recorded transitions, not correctness on every reachable state; some evidence comes from earlier components (explorer, earlier chain) or synthetic games built by us.

Availability. Linked submission: ARC-AGI-3 Milestone 2 Solution, ROP5 version 9 (`scriptVersionId` 364167125; 22.55); the unchanged RO comparison is `scriptVersionId` 363160941. Code, data and preregistrations: github.com/Baracoda-ai-labs/arc-agi-3-oracle-ladder (commit `40135bc`), with a table mapping each claim to its files; the language, the PuzzleScript interpreter and corpus results, the exact programs, the engine, the ROP5 gate, the sufficiency detector, the goal-directed search and per-run results. Our code is MIT-0; third-party code keeps its licence and unlicensed code is linked, not redistributed.

8. Conclusion

A symbolic engine that learns and plans in a formal language won levels where it was built — efficiently, cheaply and verifiably — and none where it was not; source-derived rules left our planners below an LLM agent; and supplying recognition moved the failure downstream instead of producing wins. The stage-wise audit turns these results into a map: the engine breaks at recognition, before reasoning starts, and repairs must be judged by levels won, not by better predictions. Frontier systems may benefit from stronger perceptual and representational capabilities that give downstream reasoning more informative state descriptions; we do not test that directly. For local and verifiable model-based agents, our results support a design rule: validate recognition independently before investing in transition learning or search — a model-based agent should not be optimised downstream until its representation of the environment has been shown to be adequate.

Appendix A — Soundness of deduced independences

Model each action a as a step function f_a with read set $R(a)$ and write set $W(a)$; by inertia, f_a changes nothing outside $W(a)$ and what it writes depends only on $R(a)$.

- *Lemma.* If $W(a) \cap R(b) = W(b) \cap R(a) = W(a) \cap W(b) = \varnothing$, then $f_b \circ f_a = f_a \circ f_b$: a fluent in $W(a)$ gets a value computed from $R(a)$, which b never writes, in either order; likewise for $W(b)$; other fluents are untouched.

- *Shared counters.* A fluent written by both is allowed only if each applies $x \leftarrow x \oplus \delta$ with δ independent of x and of the other's writes, and $\oplus$ commutative and associative (the move budget with $\delta = -\text{cost}$; cyclic attributes in $\mathbb{Z}/n$).
- *Budget threshold.* Defeat is read once per step and costs are non-negative: if the reserve after both moves stays above the threshold, both orders are legal and reach the same state; otherwise the pair is not reordered.
- *Autonomous effects* (clock T): apply the lemma to (a, b), (a, T) and (b, T); guards and terminal conditions belong to *R*.
- *Assumption:* declared sets over-approximate the true ones. A first version with a missing read gave 56 false independences; the corrected language gives 0 of 311.

Appendix B — Held-out games, per passage (native score; levels)

Game	RO P1	RO P2	ROP5 P1	ROP5 P2
vc33	75.00; 6/7	53.26; 5/7	100.00; 7/7	100.00; 7/7
tu93	44.06; 6/9	66.22; 8/9	67.72; 8/9	66.59; 8/9
s5i5	27.78; 4/8	41.67; 5/8	77.78; 7/8	27.78; 4/8
cn04	47.62; 4/6	47.62; 4/6	4.76; 1/6	4.76; 1/6
wa30	13.27; 3/9	13.33; 3/9	2.22; 1/9	3.37; 2/9
Mean	**41.55**	**44.42**	**50.50**	**40.50**

References

- Akyürek, E. et al. (2024). The Surprising Effectiveness of Test-Time Training for Abstract Reasoning. arXiv:2411.07279.
- ARC Prize (2026). ARC Prize 2026: ARC-AGI-3 Milestone Prize #1. arcprize.org.
- ARC Prize Foundation (2026). ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. arXiv:2603.24621.
- Banwait, H. S. (2026). Transforming Integer Grids to Spatial Intuition: Perception, Not Planning, Governs ARC-AGI-3. arXiv:2611.06793.
- Battleday, R. M., Sandbrink, K. et al. (2026). DiG-bench: Discovery in Games. arXiv:2608.12593.
- Chollet, F. (2019). On the Measure of Intelligence. arXiv:1911.01547.
- Courtis, D., Li, W., Sanner, S. (2026). OPINE-World. arXiv:2607.01531.
- Cropper, A., Evans, R., Law, M. (2019). Inductive general game playing. arXiv:1906.09627.
- Eriksson, S., Röger, G., Helmert, M. (2017). Unsolvability certificates for classical planning. ICAPS.
- Evans, R. et al. (2019). Making sense of sensory input. arXiv:1910.02227.
- *How Humans Learn Unfamiliar Worlds: A Think-Aloud Dataset on ARC-AGI-3* (2026). arXiv:2610.34523.
- Lehmann, J., Aioanei, A., Vahdati, S. (2026). Tycho. arXiv:2607.28287.
- Lehmann, J., Vahdati, S. (2026). Can LLMs Improve Their Own Harness? A Controlled Study on ARC-AGI-3. arXiv:2610.30499.
- Liew, K. H. (2026). Explore Before You Solve: The Speed–Depth Trade-off in Epistemic Agents for ARC-AGI-3. arXiv:2605.25931.
- Love, N. et al. (2008). General Game Playing: Game Description Language Specification. Stanford.
- Pan, G., Tan, Q., Yuan, Y., Yao, A. C. (2026). Algebraic World Model: From Exploration to Proofs. arXiv:2610.40503.
- Rodionov, S. (2026a). Executable World Models for ARC-AGI-3 in the Era of Coding Agents. arXiv:2605.05138.
- Rodionov, S. (2026b). Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3? arXiv:2607.15439.
- Schaul, T. (2013). A video game description language for model-based or interactive learning. IEEE CIG.
- Skoutnev, A., Acharya, K., Longhitano, G., Udell, M., Ellis, K., Drori, I. (2026). Twin: Playing an Unknown Game with a Test-Time Digital Twin. arXiv:2608.14490.
- Thielscher, M. (2009). Answer set programming for single-player games in general game playing. ICLP.
- Tsividis, P. et al. (2021). Human-level reinforcement learning through theory-based modeling, exploration, and planning. arXiv:2107.12544.
- Wehrle, M., Helmert, M. (2012). About partial order reduction in planning and computer games. ICAPS.

- Wu, C., Passenger, J., Song, Y. (2026). Tracing the Thoughts of a Coding Agent Playing ARC-AGI-3: Lessons for Continual Learning. arXiv:2610.11450.
- Wu, W. (2026). Kepler: Auditable World Models for ARC-AGI-3. arXiv:2610.00834.
- Zweiger, A. et al. (2025). Self-Adapting Language Models. arXiv:2506.10943.